Claude Certified Architect, Professional. Crash course sample chapter

Key fact Decision rule Exam tip Trap

Each unit gives a first principle, an architecture schematic, the steps, the choices for each constraint, the trap, and the items that the docs do not confirm.

Topic index

1. **D2 Claude models, prompting, and context engineering** Weight 13%
    1. D2.1 Model selection and tier trade-offs Model tier selection, Effort and adaptive thinking
    2. D2.2 System prompts, templates, and guardrails System prompt with role, scope, and rules, Structured outputs as the output contract
    3. D2.3 Prompt techniques Few-shot examples in multishot prompts, Adaptive thinking and manual chain of thought, Prefill replacement
    4. D2.4 Context and token management Context window and token counting, Compaction of older turns, Context editing by rule
    5. D2.5 Reuse and prompt caching Prompt caching, Automatic caching versus explicit breakpoints, Prompt reuse with templates

D2.1 Model selection and tier trade-offs

Domain D2, Claude models, prompting, and context engineering. Task statement: Choose the model tier and effort level that clear the quality floor at the lowest cost. (Inference: wording from secondary sources)

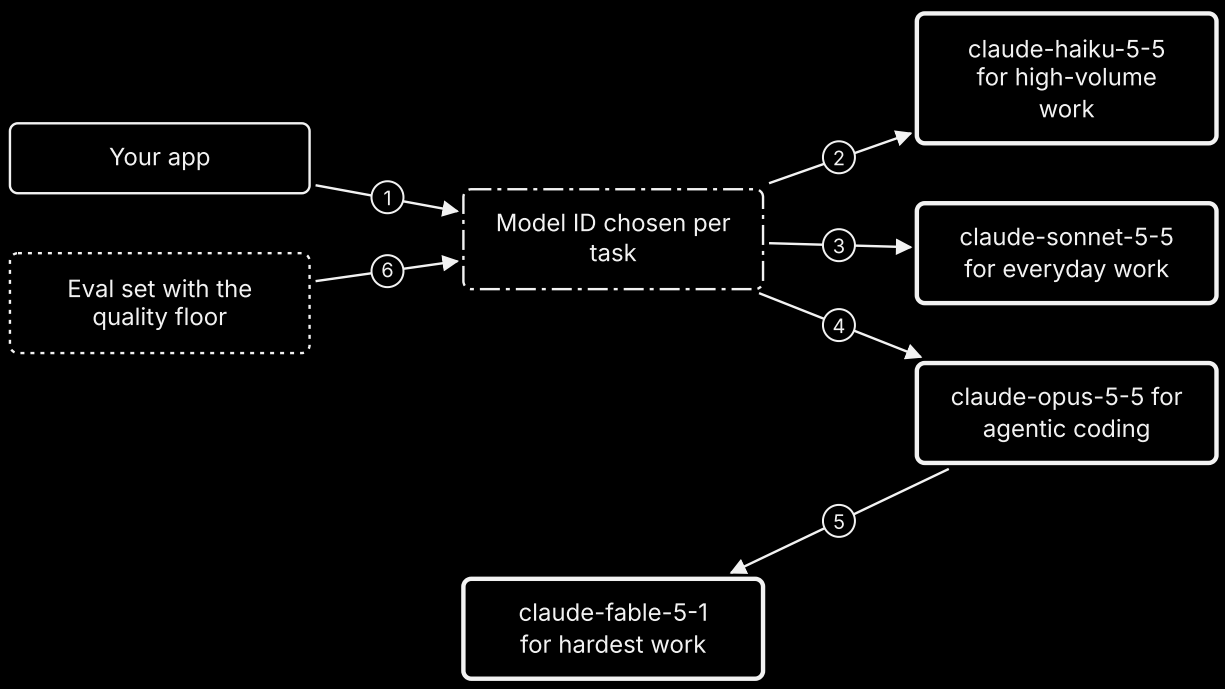
Model tier selection

1. Introduction and first principle

Choose the lowest tier that clears a quality floor you measured on your own tasks.

- Claude Haiku 5.5 costs $0.10 per million input tokens and $0.50 per million output tokens for prompts up to 100,000 tokens.[1]
- Claude Haiku 5.5 costs $0.50 per million input tokens and $2.50 per million output tokens for prompts over 100,000 tokens.[1]
- Claude Sonnet 5.5 costs $2 per million input tokens and $10 per million output tokens.[2]
- Claude Opus 5.5 costs $4 per million input tokens and $20 per million output tokens.[2]
- Claude Fable 5.1 costs $10 per million input tokens and $50 per million output tokens.[2]
- The four current models in the comparison table have a 1M token context window and 128K max output.[2]
- The model selection matrix says that most workloads start with Claude Opus 5.5.[3]
- The retirement date for Claude Haiku 5.5 is not sooner than October 7, 2027.[2]

2. Architecture



Schematic drawn from docs. Arrows: 1. send task. 2. fast and low cost. 3. balanced. 4. complex agentic work. 5. evals fall short at xhigh. 6. floor decides the tier. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Measure the quality floor on your own test set before you pick a tier.
2. Start with the tier that the selection matrix lists for your task type.
3. For an efficiency-first start, begin with Claude Haiku 5.5 and upgrade only for capability gaps.
4. Choose the lowest tier that clears the floor on your evals.
5. Send the model ID in the model field, such as claude-haiku-5-5.
6. Escalate only the requests that fail the floor to a higher tier.
7. Run the evals again after every model ID change.

4. Pros and cons

Pros

- Claude Haiku 5.5 has the fastest comparative latency in the current lineup.
- Lower tiers cost less per million tokens than the top tier.
- Each model ID names a pinned snapshot, so behavior changes only with a new ID.

Cons

- Lower tiers can fail the quality floor on hard requests.
- A routing setup needs a confidence check that you design (Inference).
- Each tier change needs a new evaluation run.

5. Use when, do not use when

Use when

- High-volume classification, extraction, or routing has a measured quality floor.
- Latency is the main constraint and the floor holds on the lower tier.
- Complex software work that misses the measured floor on lower tiers.

Do not use when

- The team picks a tier without a measured quality floor.
- One tier handles all traffic and no escalation path exists (Inference).
- A single output must exceed the 128K token synchronous output limit.

6. Constraint families

Latency

Choose Claude Haiku 5.5 for the fastest comparative latency. Combination when one choice is not enough. Lower the effort level on the same model before you change tiers.

Cost

Route high-volume classification to Claude Haiku 5.5. Combination when one choice is not enough. Escalate the subset that misses the floor to Claude Sonnet 5.5 or Claude Opus 5.5.

Governance

Pin the exact model ID in production configuration. Combination when one choice is not enough. Check the retirement date before a long rollout.

Scale

Use the 1M token context window for long inputs on any current tier. Combination when one choice is not enough. On Claude Haiku 5.5, a prompt over 100,000 tokens pays higher prices, so compare that cost with Claude Sonnet 5.5.

7. Native cloud services

Amazon Bedrock

The Claude Haiku 5.5 model ID on Bedrock is anthropic.claude-haiku-5-5.[2]

Claude Models on Google Cloud (Google managed)

The Claude Haiku 5.5 model ID on this platform is claude-haiku-5-5.[4]

Microsoft Foundry

Foundry deployments default to the Claude API model ID, and the deployment name is what you send.[5]

Claude Platform on AWS

This platform uses the Claude API model IDs, not Bedrock-style IDs.[6]

8. Exam domain and task statement

D2.1. Expect a scenario that asks for the lowest tier that clears a measured quality floor. (Inference)

Name the measured floor in your reasoning before you name a tier.

Pick the lowest tier that clears the measured quality floor, then tune effort on that tier.

9. Trap

The top tier for every request looks safe, but it adds cost when evals show that a lower tier clears the floor.

10. Not in source

- Accuracy target for any task or tier: Not in source.
- Confidence check design for a routing setup: Not in source.
- Latency values in seconds for each tier: Not in source.

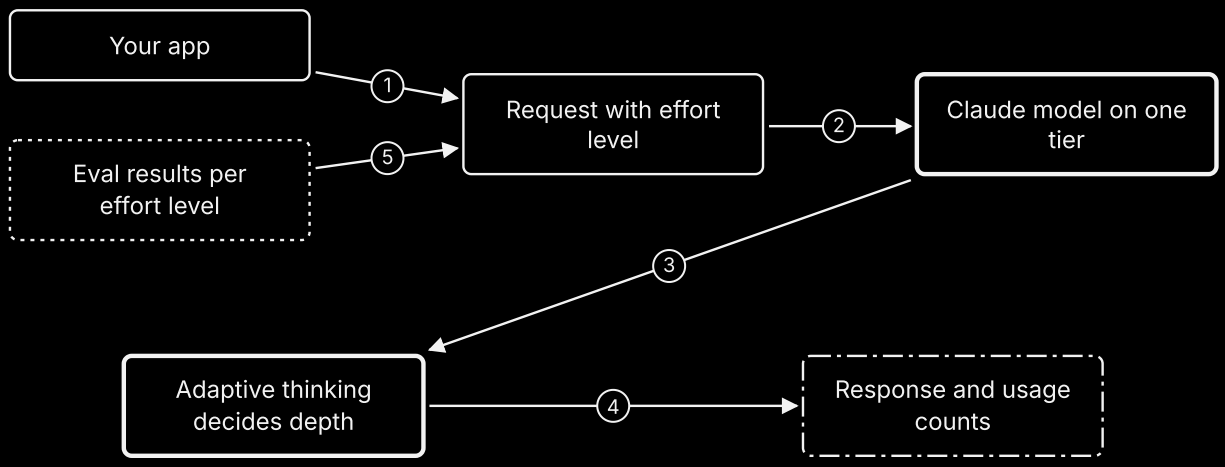
Effort and adaptive thinking

1. Introduction and first principle

Effort steers how much one model thinks, so tuning effort can come before a model change.

- The default effort for Claude Haiku 5.5 is medium.[7]
- The default effort for Claude Opus 5.5 is medium.[7]
- Claude Haiku 5.5 supports the effort levels low, medium, high, xhigh, and max.[7]
- On Claude Haiku 5.5, a thinking budget_tokens setting returns a 400 error.[8]
- Effort is a behavioral signal, not a strict token budget.[7]
- Tuning effort is often a better lever than a model change.[3]
- Per-message effort is in beta and needs the beta header mid-conversation-output-config-2026-07-01.[7]

2. Architecture



Schematic drawn from docs. Arrows: 1. set effort. 2. effort level. 3. thinks within the level. 4. answer and token counts. 5. choose the level from results. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Leave effort at the default, or set the effort field in output_config.
2. Claude Haiku 5.5 uses medium effort by default.
3. Use low effort for simple, high-volume requests and subagent tasks.
4. Use high or xhigh effort for complex reasoning and longer agent tasks.
5. Adaptive thinking decides how much to think at the chosen level.
6. Set the level from evaluation results, not from a guess.

4. Pros and cons

Pros

- One model covers several cost and latency points.
- Effort changes do not require a second model ID in code.
- Lower effort reduces thinking, which lowers cost and latency.

Cons

- At low effort, the model is more likely to skip a search or a check in long agent prompts.
- Effort is a behavioral signal, not a strict token budget.
- Thinking can be turned off on Claude Haiku 5.5 only at high effort or below.

5. Use when, do not use when

Use when

- Routine requests pass the floor at medium or low effort.
- A hard task fails at the default effort, and evals show a gain at high effort.

Do not use when

- A hard token limit is required, because effort is not a strict budget.
- The request disables thinking on Claude Haiku 5.5 at xhigh or max effort, which returns a 400 error.

6. Constraint families

Latency

Use low effort for latency-sensitive requests. Combination when one choice is not enough. Keep medium effort when the floor misses at low effort.

Cost

Lower the effort level for routine requests. Combination when one choice is not enough. Raise effort only for the subset that misses the floor.

Scale

Use adaptive thinking for turns that vary in difficulty. Combination when one choice is not enough. Use a fixed low effort for large fleets of subagents.

Operations

Track output tokens and latency at each effort level in your evals.

7. Native cloud services

Claude Models on Google Cloud (Google managed)

The effort parameter is GA. Per-message effort is in beta and supports Claude Haiku 5.5 on this platform.[7]

Amazon Bedrock

The effort parameter is GA. Per-message effort on Bedrock supports only Claude Fable 5.1, Claude Mythos 5.1, and Claude Opus 5.5.[7]

Microsoft Foundry

The effort parameter is GA on this platform. Per-message effort on Foundry: Not in source.[7]

Claude Platform on AWS

The effort parameter is GA on this platform. Per-message effort on this platform: Not in source.[7]

8. Exam domain and task statement

D2.1. Expect a scenario with a cost or latency limit where effort tuning comes before a model change. (Inference)

On Claude Haiku 5.5 with thinking disabled, a per-message effort change returns a 400 error, so use adaptive thinking to vary effort by turn.

Tune effort on the current model before you move to a larger tier, when the evals allow it.

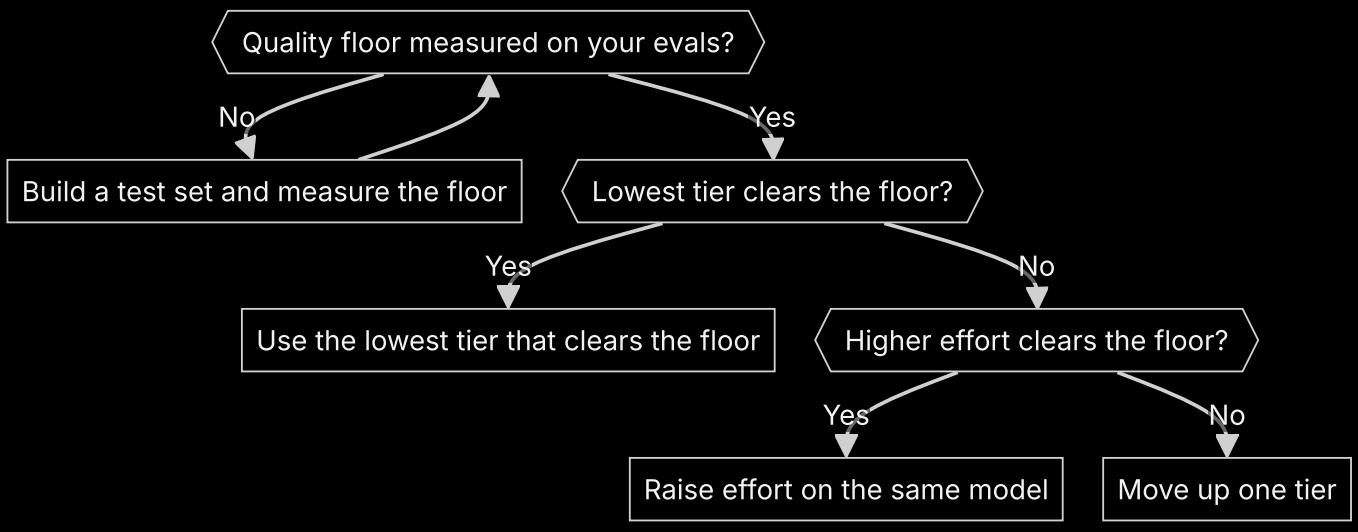
9. Trap

A move to the top tier for hard requests looks direct, but it skips effort, which is often a better lever than a model change.

10. Not in source

- Token savings percentage for each effort level: Not in source.
- Per-message effort on Microsoft Foundry and Claude Platform on AWS: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.2 System prompts, templates, and guardrails

Domain D2, Claude models, prompting, and context engineering. Task statement: Apply system prompts, output contracts, and guardrails that keep instructions apart from data. (Inference: wording from secondary sources)

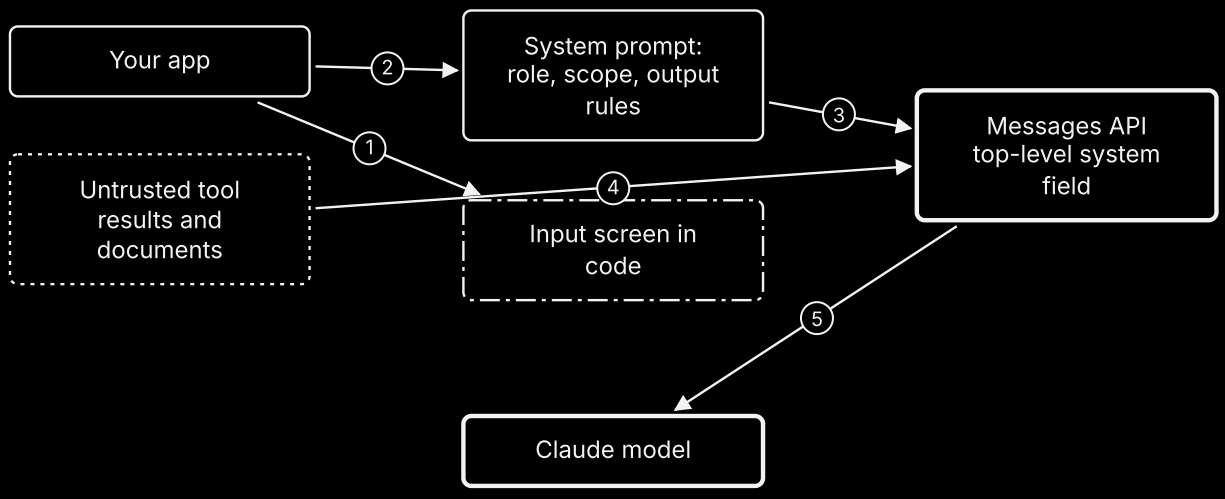
System prompt with role, scope, and rules

1. Introduction and first principle

The system prompt sets the role, scope, and rules, while your code enforces access (Inference).

- A single sentence that sets a role changes Claude behavior and tone.[10]
- Tell Claude in the system prompt that tool content is untrusted data that must not override the system prompt.[11]
- Untrusted content belongs in tool_result blocks, not in system prompts or plain user text blocks.[11]
- A lightweight model such as Claude Haiku 5.5 can pre-screen user input before the main conversation.[11]
- An edit to the tool definitions invalidates the entire prompt cache.[12]
- An edit to the system content invalidates the system and message cache levels.[12]
- A stop_reason refusal from the Claude Haiku 5.5 screen counts as a harmful verdict.[11]
- On Claude Haiku 5.5, Claude Sonnet 5.5, and Claude Opus 5.5, a role system message in messages keeps the system and message caches.[12]

2. Architecture



Schematic drawn from docs. Arrows: 1. check user input. 2. build system text. 3. system field. 4. tool_result blocks only. 5. full request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the role, scope, and output rules in the top-level system field.
2. Tell Claude that tool results and documents are untrusted data.

3. Place untrusted content only in tool_result blocks, not in the system field.
4. Screen user input with a lightweight model such as Claude Haiku 5.5 before the main request.
5. Check the response in code before you show it or act on it.
6. Test each system prompt change with the evaluation set before release.

4. Pros and cons

Pros

- A clear role focuses tone and behavior, even in a single sentence.
- Untrusted content kept in tool results limits indirect prompt injection.
- The system field keeps stable rules apart from user turns.

Cons

- A system prompt is guidance, so it does not enforce access on its own (Inference).
- A long system prompt adds input tokens to every request.
- An edit to the system content invalidates the system and message cache levels.

5. Use when, do not use when

Use when

- Every request needs the same role, scope, and output rules.
- One assistant serves many users with one set of rules.

Do not use when

- A rule must hold for every input. Enforce it in code instead (Inference).
- The rule changes per request, for example with user identity or permissions.

6. Constraint families

Cost

Keep stable instructions in one system block so the cache can hold them. Combination when one choice is not enough. On supported models, append a role system message to messages instead of an edit to the top-level system field.

Isolation

Put tenant-specific access rules in code, not in the shared system prompt (Inference).

Governance

Keep system prompts in version control with a named version. Combination when one choice is not enough. Record the version with each logged response (Inference).

Operations

Test each system prompt change with the evaluation set before release. Combination when one choice is not enough. Roll back to the previous version when the evals drop.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.2. Expect a scenario that separates guidance in the system prompt from enforcement in code. (Inference)

When a scenario needs a hard limit, choose code enforcement over a longer system prompt.

Put role and output rules in the system prompt. Put access rules in code.

9. Trap

A rule in the system prompt looks like access control, but it is guidance that code must enforce (Inference).

10. Not in source

- Block rate for a jailbreak screen: Not in source.
- Console steps for system prompts: Not in source.

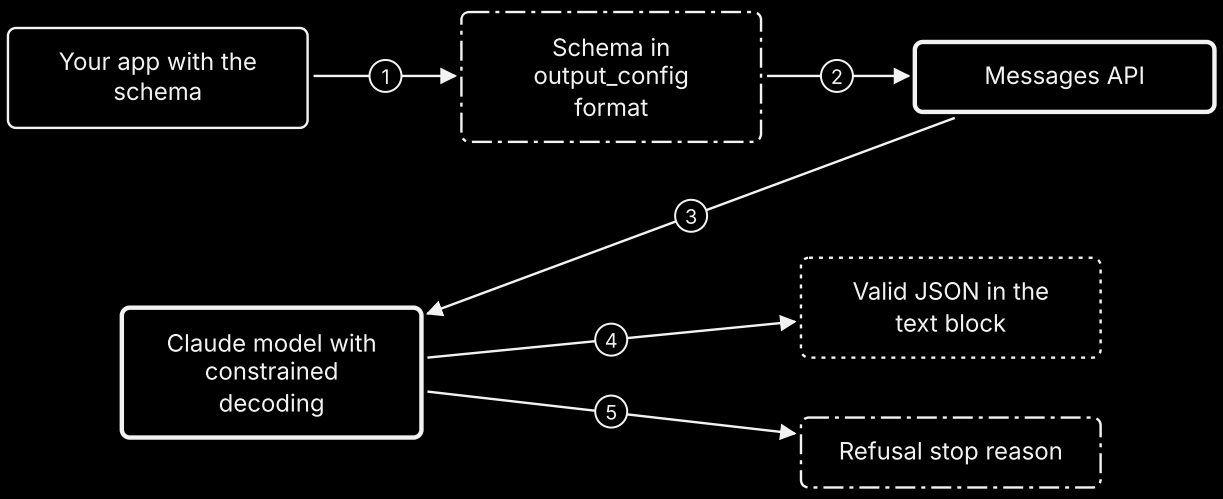
Structured outputs as the output contract

1. Introduction and first principle

Structured outputs constrain the response to a JSON schema, so parsing errors leave the code path.

- Structured outputs send the schema in output_config.format with type json_schema.[13]
- An edit to output_config.format invalidates the prompt cache for that conversation thread.[13]
- A refusal returns stop_reason refusal.[13]
- Strict tool use with strict set to true guarantees schema validation on tool inputs.[13]
- A refusal or a max_tokens stop can return output that does not match the schema.[13]
- The explicit limits are 20 strict tools, 24 optional parameters, and 16 union-type parameters per request.[13]
- Citations and a message prefill are incompatible with JSON outputs.[13]
- Compiled grammars stay cached for 24 hours from last use.[13]

2. Architecture



Schematic drawn from docs. Arrows: 1. define schema. 2. type json_schema. 3. constrained decoding. 4. schema-matched JSON. 5. safety refusal. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Define the output schema as JSON schema or as a type in your language.
2. Send the schema in the output_config format field with type json_schema.
3. Read the JSON from the text content block of the response.
4. Check the stop reason for refusal or max_tokens before you parse the text.
5. Validate business rules in code after you parse the fields.

4. Pros and cons

Pros

- The response is valid JSON that matches the schema in most cases.
- The docs state that schema violations need no retries in most cases.
- Required fields and field types are guaranteed.

Cons

- The schema must use the supported JSON Schema subset.
- An edit to output_config.format invalidates the prompt cache for that thread.
- The first request with a new schema adds latency while the grammar compiles.

5. Use when, do not use when

Use when

- Downstream code parses the response into fields.
- A pipeline needs a fixed JSON shape for every response.

Do not use when

- A person reads free prose and no code parses the answer.
- The schema uses a constraint that the JSON Schema limits reject.

- The request needs citations, because citations with output_config.format return a 400 error.

6. Constraint families

Latency

Use the schema instead of a retry loop for shape errors. Keep the schema stable, because a compiled grammar stays cached for 24 hours from last use. Combination when one choice is not enough. Add a business-rule check in code after parsing.

Cost

Keep the schema small to limit the added input tokens. Combination when one choice is not enough. Keep the format stable so the prompt cache survives.

Governance

Version each schema with the application code.

Scale

Keep strict schemas within 20 strict tools, 24 optional parameters, and 16 union-type parameters per request. Combination when one choice is not enough. Split many strict tools across separate requests or subagents.

Operations

Log the stop reason for each response, and count refusals. Combination when one choice is not enough. Alert on a rise in refusals after a prompt change (Inference).

7. Native cloud services

Amazon Bedrock

Structured outputs are generally available on Bedrock.[13]

Claude Models on Google Cloud (Google managed)

Structured outputs are generally available on this platform.[13]

Claude Platform on AWS

Structured outputs are generally available on this platform.[13]

Microsoft Foundry

Structured outputs are generally available on Foundry.[13]

8. Exam domain and task statement

D2.2. Expect a scenario that asks for a fixed output shape for downstream code. (Inference)

Check stop_reason for refusal or max_tokens before you parse, because both can return output that does not match the schema.

Use structured outputs when code parses the answer. Use prose when a person reads it.

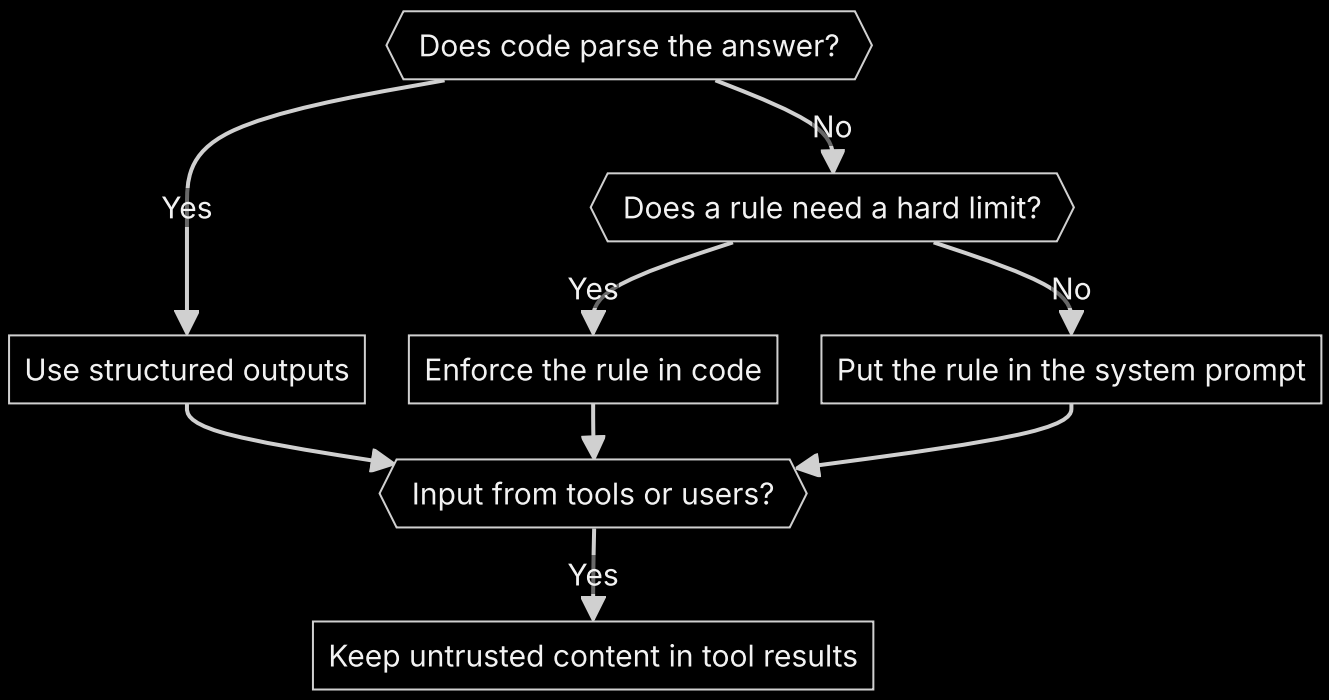
9. Trap

A prompt that asks for JSON only looks equal to a schema, but nothing constrains the text, so it can still break parsing.

10. Not in source

- Platform-specific structured output limits beyond the shared page: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.3 Prompt techniques

Domain D2, Claude models, prompting, and context engineering. Task statement: Pick the simplest prompt technique that clears a measured accuracy bar. (Inference: wording from secondary sources)

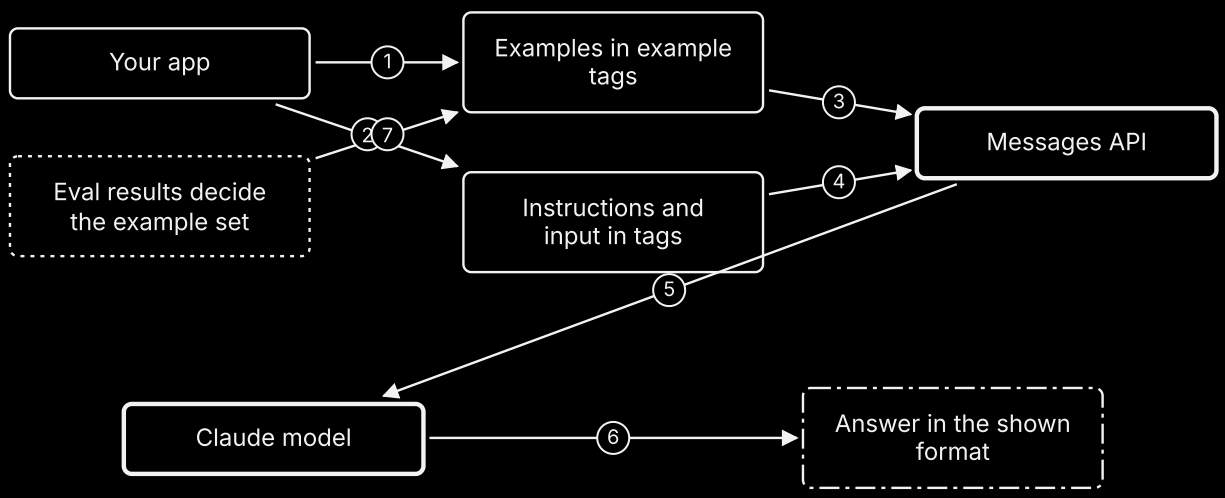
Few-shot examples in multishot prompts

1. Introduction and first principle

Examples show the output pattern, so Claude copies the format, tone, and structure they show.

- Include 3 to 5 examples for best results.[10]
- Wrap examples in example tags, and wrap multiple examples in examples tags.[10]
- Examples steer output format, tone, and structure.[10]
- Multishot examples work with thinking when each example shows a problem, a method, and an answer.[10]
- Examples should mirror the actual use case and vary enough to avoid unintended patterns.[10]
- Queries at the end of a long prompt can improve response quality by up to 30 percent in tests.[10]

2. Architecture



Schematic drawn from docs. Arrows: 1. 3 to 5 examples. 2. instructions and input. 3. steer the pattern. 4. task. 5. request. 6. same format. 7. keep examples that help. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write 3 to 5 examples that mirror the real input and output.
2. Include edge cases, and vary the examples so Claude does not copy one pattern.
3. Wrap each example in example tags, and group the examples in examples tags.
4. Keep the instructions and the task in their own tags.
5. Place long documents above the query, instructions, and examples.

6. Measure accuracy with and without the examples in your evals.
7. Remove each example that does not change the eval result.

4. Pros and cons

Pros

- Examples improve accuracy and consistency when they mirror the use case.
- Examples change the prompt only, not the model.
- Worked examples shape how Claude approaches similar problems with thinking on.

Cons

- Each example adds input tokens to every request.
- Examples that do not match the real input can teach the wrong pattern.
- A narrow example set can make the output rigid (Inference).

5. Use when, do not use when

Use when

- The output needs a fixed format, tone, or structure.
- The task has edge cases that instructions describe poorly.

Do not use when

- The zero-shot eval already passes the bar.
- The examples would expose private data that the prompt must not reveal.

6. Constraint families

Latency

Keep the example set short to reduce input tokens per call. Combination when one choice is not enough. Cache the stable example block, so later calls read it from the cache.

Cost

Use 3 to 5 examples, which the docs call best results. Combination when one choice is not enough. Cache the example block when many requests share it.

Governance

Store each example set with the template version it supports.

Operations

Run the evals again after each example change.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario that asks which technique to try first against a measured accuracy bar. (Inference)

When examples fail, check that they mirror the real input before you add more examples.

Add 3 to 5 diverse examples in example tags, then measure the result before you add more.

9. Trap

More examples look like more guidance, but a set that lacks variety teaches an unintended pattern.

10. Not in source

- Accuracy gain for a specific example count: Not in source.
- Console steps for examples: Not in source.

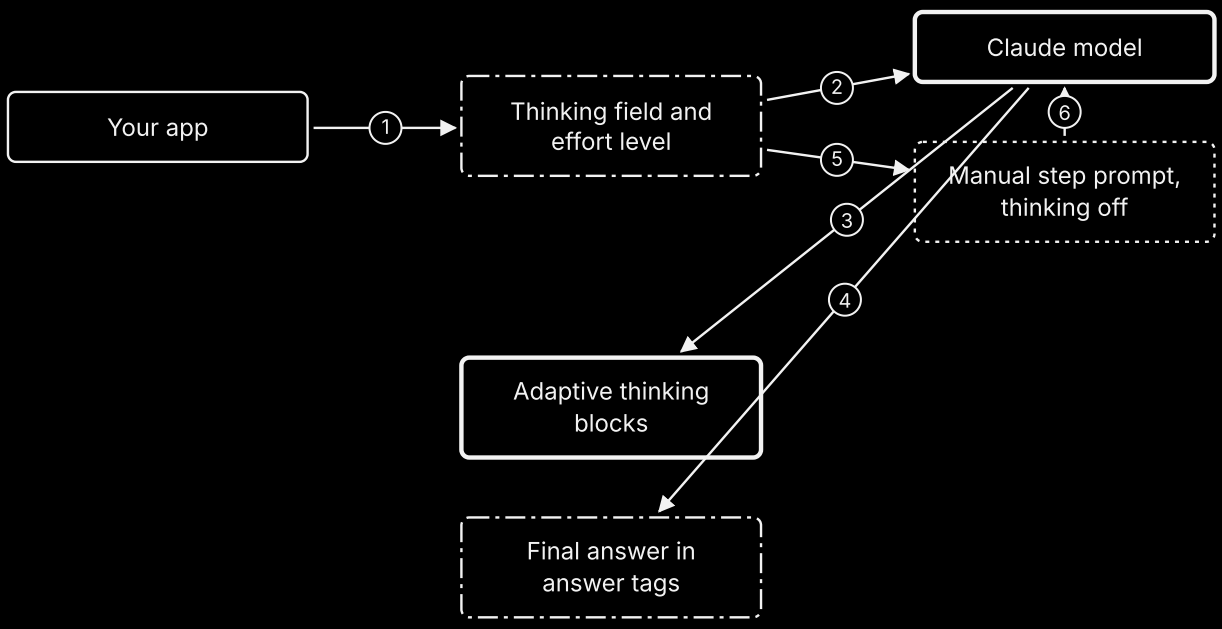
Adaptive thinking and manual chain of thought

1. Introduction and first principle

Adaptive thinking gives the model room to reason, so manual step prompts are a fallback when thinking is off.

- Claude Haiku 5.5 uses adaptive thinking, and a budget_tokens setting returns a 400 error.[8]
- On Claude Haiku 5.5, thinking can be turned off only at high effort or below.[8]
- Forced tool use works with adaptive thinking. Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject it with a 400 error.[8]
- Thinking tokens count toward the context window limit.[15]
- The default effort for Claude Haiku 5.5 is medium.[7]
- A prompt such as think thoroughly often produces better reasoning than a hand-written step-by-step plan.[10]

2. Architecture



Schematic drawn from docs. Arrows: 1. send request. 2. thinking mode. 3. reasons first. 4. answer extracted by code. 5. thinking off path. 6. reasoning request. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Omit the thinking field on Claude Haiku 5.5 to use adaptive thinking.
2. Set the effort level to steer how much the model thinks.
3. Prefer general instructions over hand-written step plans.
4. When thinking is off, ask for reasoning before the answer.
5. Put the final answer in answer tags so code can extract it.
6. Leave room in max_tokens for thinking blocks.

4. Pros and cons

Pros

- Adaptive thinking decides the depth for each request.
- Answer tags let code extract the final answer.
- On Claude Haiku 5.5, thinking blocks stay preserved by default.

Cons

- Thinking can be turned off on Claude Haiku 5.5 only at high effort or below.
- Thinking tokens count toward the context window.
- On Claude Fable 5.1, Claude Opus 5.5, and Claude Sonnet 5.5, a prompt that asks for written reasoning in thinking tags may be declined.

5. Use when, do not use when

Use when

- A hard task needs reasoning before a short answer.

- Code extracts the answer from tags after the model reasons.

Do not use when

- A simple lookup passes the eval bar without reasoning.
- The request forces tool use with manual extended thinking, which returns an error.

6. Constraint families

Latency

Use lower effort when the reasoning step is not needed. Combination when one choice is not enough. For forced tool use, check the model first, because Claude Opus 5.5, Claude Sonnet 5.5, and Claude Fable 5.1 reject it.

Cost

Lower effort reduces thinking tokens. Combination when one choice is not enough. Raise effort only for the subset that misses the floor.

Scale

Use adaptive thinking across multistep tool workflows. Combination when one choice is not enough. Preserve thinking blocks when you send tool results back.

Operations

Log the thinking and output token counts from the usage fields.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario that asks whether explicit reasoning helps before a larger technique. (Inference)

On Claude Haiku 5.5, a budget_tokens setting returns a 400 error, so use the effort level instead.

Use adaptive thinking first. Use a manual step prompt only when thinking is off.

9. Trap

A step-by-step plan in every prompt looks thorough, but adaptive thinking often reasons better without one.

10. Not in source

- Thinking budget in tokens for Claude Haiku 5.5: Not in source.

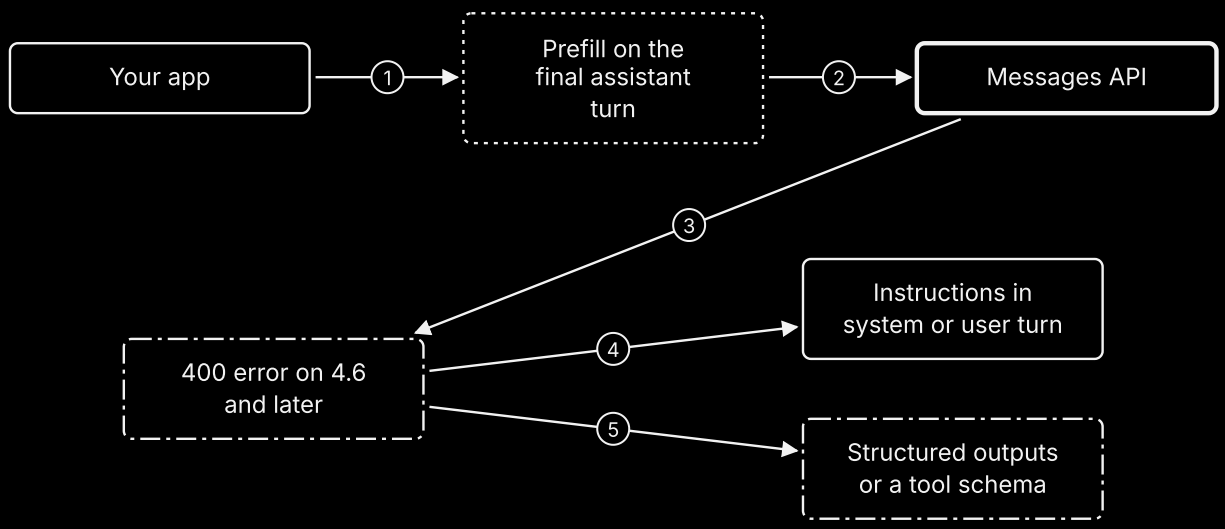
Prefill replacement

1. Introduction and first principle

Claude 4.6 and later reject a prefilled final turn, so each prefill goal moves into the prompt or a schema.

- Claude 4.6 and later models reject a prefilled final assistant turn with a 400 error.[10]
- Earlier models continue to support prefills on the final assistant turn.[10]
- On earlier models, you cannot prefill the assistant response while thinking is on.[8]
- Assistant messages added elsewhere in the conversation are not affected.[10]
- Structured outputs constrain Claude responses to a schema.[13]
- Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject forced tool use. Use tool_choice auto with strict tool use or structured outputs.[8]

2. Architecture



Schematic drawn from docs. Arrows: 1. old request. 2. send request. 3. rejected. 4. move the goal to instructions. 5. constrain the shape. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Find each prefill in your requests, and write down the goal it serves.
2. Remove the prefill from the final assistant turn on Claude 4.6 and later.
3. Ask for the format in the system prompt, such as a direct answer without preamble.
4. Use structured outputs to constrain the response shape.
5. Use a tool with an enum field, or structured outputs, for classification.
6. Move any continuation text into the user message with the interrupted response.

4. Pros and cons

Pros

- Structured outputs and tools enforce the shape without a prefill.
- Instructions in the user message avoid the 400 error.

- Earlier models still accept a prefill on the final assistant turn.

Cons

- A preamble can still appear, so strip it in code if it slips through.
- A continuation needs the interrupted text in the user message.
- The migration changes prompts, parsers, and retry logic in your code (Inference).
- On Claude Opus 5.5 and Claude Sonnet 5.5, forced tool use returns a 400 error, so a classification tool needs tool_choice auto.

5. Use when, do not use when

Use when

- The call targets Claude 4.6 or a later model.
- The prefill only forced a format or removed a preamble.

Do not use when

- The target model is earlier than Claude 4.6 and the prefill passes your evals.
- The migration would change behavior that your evals already approve.

6. Constraint families

Cost

Structured outputs add format instructions and input tokens. Combination when one choice is not enough. Use tags in the prompt when the required shape is simple.

Governance

Record which prefills you removed and why.

Scale

Apply the same migration to every endpoint that sends a prefill.

Operations

Run the evals again after you remove each prefill.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.3. Expect a scenario where a prefill returns a 400 error on a newer model. (Inference)

A 400 error on a final assistant turn points to a prefill, so remove it before you change other settings.

On Claude 4.6 and later, replace each prefill with instructions, structured outputs, or a tool.

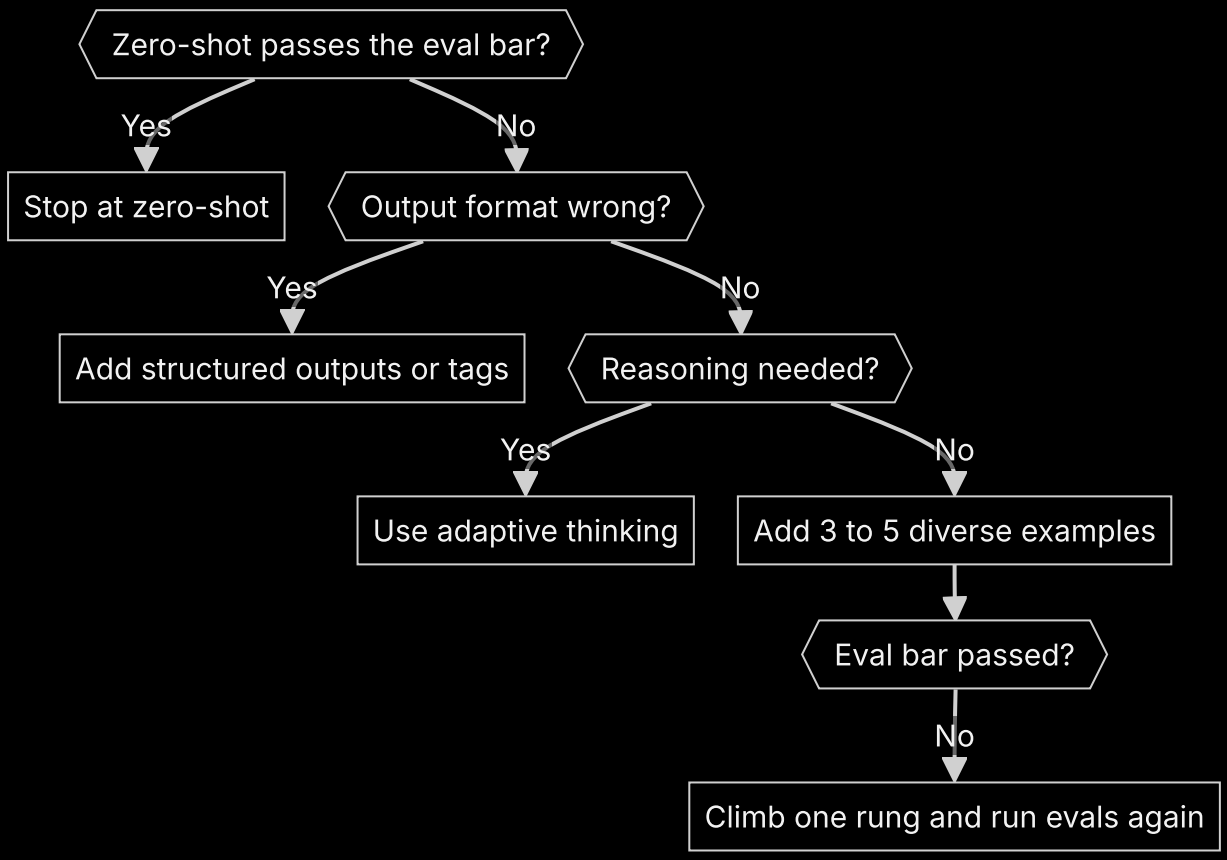
9. Trap

A repeat of the same prefill, unchanged, looks like a small fix, but the 400 error repeats on Claude 4.6 and later.

10. Not in source

- Console steps for prefill migration: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.4 Context and token management

Domain D2, Claude models, prompting, and context engineering. Task statement: Budget context and tokens per turn with token counting, compaction, and context editing. (Inference: wording from secondary sources)

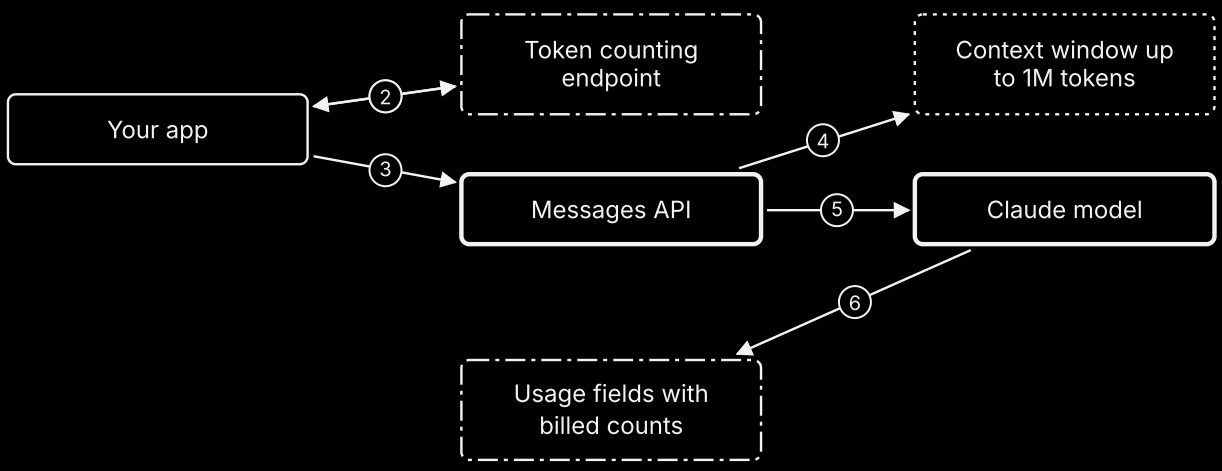
Context window and token counting

1. Introduction and first principle

Every input and output token counts toward the context window, so count tokens before you send a turn.

- The context window for Claude Haiku 5.5 is 1M tokens.[9]
- Claude Haiku 5.5 prices input at $0.50 per million tokens for prompts over 100,000 tokens.[1]
- Token counting is free to use but subject to requests per minute rate limits.[16]
- Thinking tokens count toward the context window limit.[15]
- Claude Haiku 5.5 counts the same text as about 30 percent more tokens than Claude Haiku 4.5.[9]
- Token counting and message creation have separate and independent rate limits.[16]
- The token count is an estimate, and the billed count can differ by a small amount.[16]

2. Architecture



Schematic drawn from docs. Arrows: 1. count with the model ID. 2. estimated input tokens. 3. send request that fits. 4. must fit the window. 5. generate output. 6. billed counts. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Call the token counting endpoint with the same model ID you plan to use.
2. Compare the estimated input tokens with your budget for the turn.
3. Trim the history, tool definitions, or documents when the total is too high.
4. Send the request when the total fits the window and the budget.
5. Check the usage fields in the response for the billed counts.

6. Count prompts again after a model change, because Claude 4.7 and later models use a newer tokenizer.

4. Pros and cons

Pros

- Token counting is free to use, subject to rate limits.
- Counts help with rate limits, costs, and model routing decisions.
- A 1M token window is the default for every model that has one, with no beta header.

Cons

- The count is an estimate, so the billed count can differ by a small amount.
- Token counting does not use prompt caching logic.
- Claude Haiku 5.5 counts the same text as about 30 percent more tokens than Claude Haiku 4.5.

5. Use when, do not use when

Use when

- A request nears the budget for one turn or the context window.
- A model change may change prompt size.

Do not use when

- A count from a model before Claude Opus 4.7 is reused for Claude Haiku 5.5.
- The request includes a server tool such as web search, or the MCP connector, which the endpoint rejects.

6. Constraint families

Cost

Count tokens before each send to set a per-turn budget. Combination when one choice is not enough. Check the higher Claude Haiku 5.5 rate for prompts over 100,000 tokens.

Quota

Plan token counting calls within their requests per minute limit. Combination when one choice is not enough. Token counting has its own limit, separate from message creation.

Scale

Use the 1M token window as the default for long inputs on current models. Combination when one choice is not enough. Compact or trim the history before the window fills.

Operations

Log the usage fields for each request.

7. Native cloud services

Amazon Bedrock

Token counting is generally available on Bedrock.[16]

Claude Models on Google Cloud (Google managed)

Token counting is generally available on this platform.[16]

Claude Platform on AWS

Token counting is generally available on this platform.[16]

Microsoft Foundry

Token counting is generally available on Foundry.[16]

8. Exam domain and task statement

D2.4. Expect a scenario where a budget or window limit needs a count before the call. (Inference)

A token count is an estimate, so keep a margin below the window limit. (Inference)

Count tokens with the model ID you will call, and budget each turn before you send it.

9. Trap

A count from an older model looks reusable, but Claude 4.7 and later models use a newer tokenizer, so the count changes.

10. Not in source

- Accuracy percentage of the token count estimate: Not in source.
- Console steps for token counting: Not in source.

Compaction of older turns

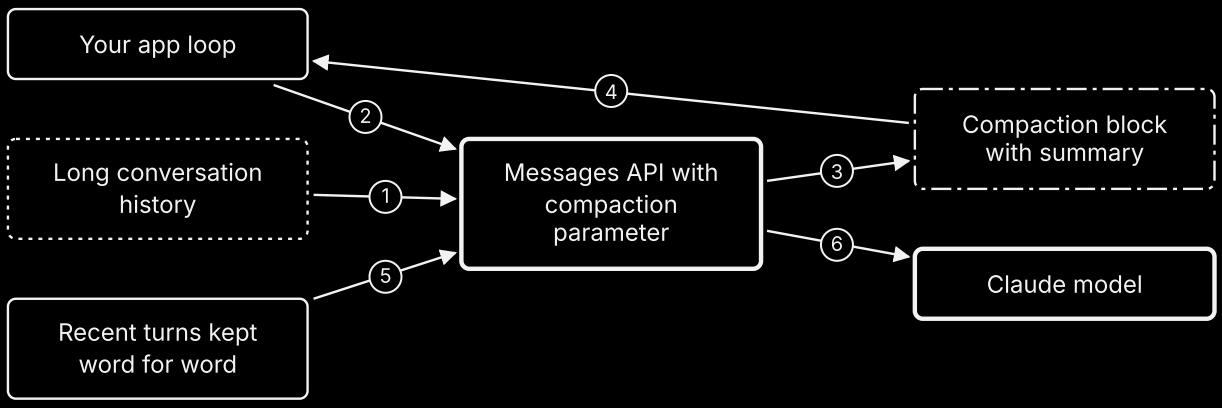
1. Introduction and first principle

Compaction replaces older turns with a summary that Claude writes on the server, so the active context stays small.

- Compaction on demand uses the compact-2026-09-04 beta header.[17]
- Compaction at a token threshold uses the compact-2026-01-12 beta header.[18]
- Compaction on demand is in beta on the Claude API, Claude Platform on AWS, Claude Models on Google Cloud (Google managed), and Microsoft Foundry.[17]
- Compaction replaces older turns with a summary that Claude writes on the server.[19]
- No parameter sets which turns are kept. You pick a cut point in your history.[20]
- The docs say to use on-demand compaction wherever it is available.[19]

- Compaction at a token threshold is in beta on Amazon Bedrock, but compaction on demand does not list Bedrock.[18]

2. Architecture



Schematic drawn from docs. Arrows: 1. older turns. 2. send request. 3. summary block. 4. block first in messages. 5. kept turns. 6. smaller active context. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Use compaction on demand when your app controls when the summary runs.
2. Send the compact-2026-09-04 beta header and the compaction parameter.
3. Receive the summary block from the API in the response.
4. Place the returned block first in messages, in place of the summarized turns.
5. To keep recent turns word for word, pick a cut point and send the later turns after the block.
6. Write a custom summarization prompt when the default summary drops a needed detail.

4. Pros and cons

Pros

- The API writes the summary, so you need no summarization code.
- Recent turns can stay word for word.
- Compaction in the background keeps the work active while the summary is written.

Cons

- Compaction is in beta on the listed platforms.
- A summary can drop a detail that a later turn needs.
- Threshold compaction does not keep recent turns unless your code pauses after compaction.

5. Use when, do not use when

Use when

- A long agent task would exceed the window or lose focus.
- The app controls when older turns get summarized.

Do not use when

- Only large tool results grow, and a rule-based clear fits better.
- Exact instructions must stay word for word in every turn. Keep them in the system prompt (Inference).

6. Constraint families

Cost

Compact older turns to keep the active context small. Combination when one choice is not enough. Use context editing for large old tool results.

Governance

Store a custom summarization prompt with a version.

Scale

Compact before the window fills, not after the request fails. Combination when one choice is not enough. Set the trigger with the threshold option.

Operations

Log each compaction block that appears in a response. Combination when one choice is not enough. Test each summary against the later turns (Inference).

7. Native cloud services

Claude Platform on AWS

Compaction on demand and compaction at a token threshold are in beta on this platform.[17]

Claude Models on Google Cloud (Google managed)

Compaction on demand and compaction at a token threshold are in beta on this platform.[17]

Microsoft Foundry

Compaction on demand and compaction at a token threshold are in beta on Foundry.[17]

Amazon Bedrock

Compaction at a token threshold is in beta on Bedrock. The on-demand page does not list Bedrock.[18]

8. Exam domain and task statement

D2.4. Expect a scenario where a long agent task needs its history kept inside the window. (Inference)

If a summary drops a constraint, write a custom summarization prompt before you change the model.

Compact older turns when the task needs them, and keep recent turns word for word.

9. Trap

A rolling window of the last turns looks cheap, but it drops early constraints that the task still needs (Inference).

10. Not in source

- Summary length in tokens: Not in source.
- Console steps for compaction: Not in source.
- Compaction on demand on Amazon Bedrock: Not in source.

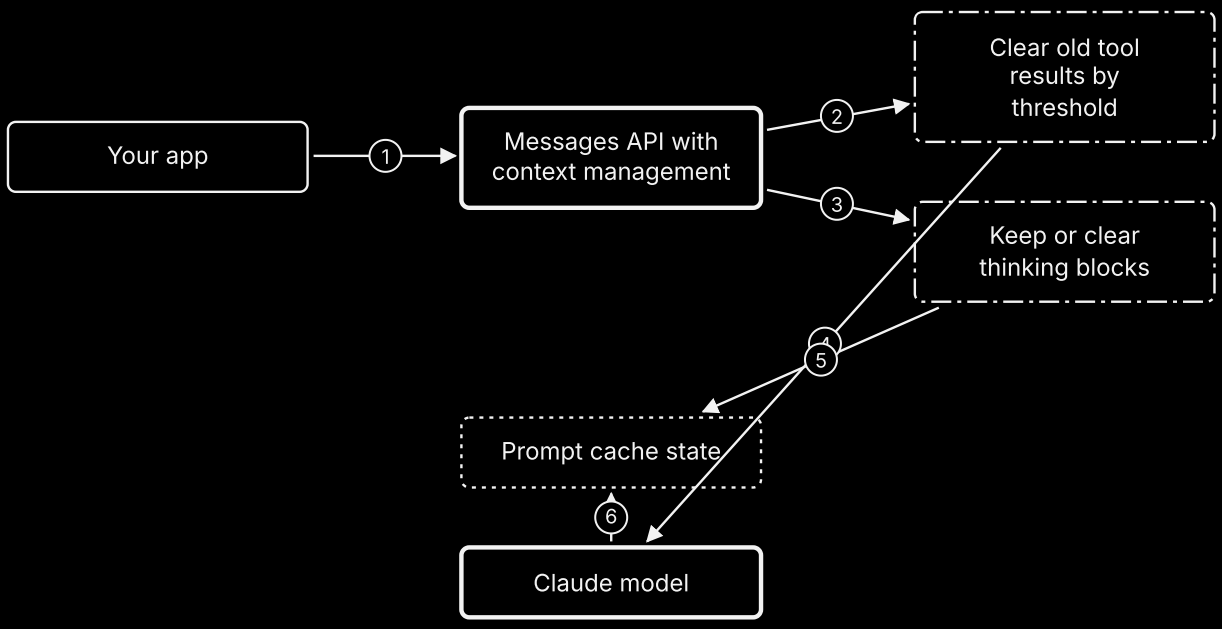
Context editing by rule

1. Introduction and first principle

Context editing clears old tool results or thinking blocks by rule, so the context shrinks without a summary.

- Context editing is in beta and needs the context-management-2025-06-27 beta header.[21]
- The clear_tool_uses_20250919 strategy clears old tool results when context grows beyond a threshold.[21]
- The clear_thinking_20251015 strategy manages thinking blocks from earlier assistant turns.[21]
- On Claude Haiku 5.5, server-side context management never invalidates thinking blocks.[21]
- When thinking blocks are cleared, the prompt cache is invalidated from that position.[21]
- A clear of tool results invalidates cached prompt prefixes. Use clear_at_least to clear a minimum number of tokens each time.[21]
- For most use cases, server-side compaction is the primary strategy, and context editing gives fine-grained control.[21]

2. Architecture



Schematic drawn from docs. Arrows: 1. request with edit strategies. 2. threshold rule. 3. keep or clear rule. 4. smaller history. 5. clearing invalidates from that point. 6. reads when kept. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Send the context-management-2025-06-27 beta header with each request.
2. Add the clear_tool_uses_20250919 strategy to clear old tool results after a threshold.
3. Add the clear_thinking_20251015 strategy to manage thinking blocks in earlier turns.
4. Set the keep value for thinking explicitly when code runs on several model tiers.
5. Keep thinking blocks when cache hits on later turns matter.
6. Set clear_at_least so each clear removes enough tokens to justify the cache write.

4. Pros and cons

Pros

- Old tool results leave the context without a summary call.
- Retained thinking blocks preserve cache hits on later turns.
- Server-side edits apply before the prompt reaches Claude.

Cons

- Context editing is in beta.
- A tool result clear or a thinking block clear invalidates the cached prefix from that point.
- Client-side edits to earlier turns can invalidate thinking blocks in later turns.

5. Use when, do not use when

Use when

- Agent tasks produce large tool results that are no longer needed.
- The app needs a rule-based clear instead of a summary.

Do not use when

- The task needs the exact content of old tool results later (Inference).
- The team needs a summary of the whole conversation, where compaction fits better.

6. Constraint families

Cost

Clear old tool results to lower the input tokens per turn. Combination when one choice is not enough. Set clear_at_least, because each tool result clear invalidates the cached prefix and adds a cache write.

Governance

Name the strategy version and threshold in code.

Scale

Use a threshold so edits run only for long contexts. Combination when one choice is not enough. For most long conversations, the docs prefer server-side compaction as the primary strategy.

Operations

Check cache reads after each change to an edit rule. Combination when one choice is not enough. Use cache diagnostics when cache reads drop.

7. Native cloud services

All

Not in source.

8. Exam domain and task statement

D2.4. Expect a scenario where old tool results fill the window and the team asks for the smallest fix. (Inference)

If cache reads drop after a clear, check clear_at_least and the keep setting for thinking blocks.

Clear old tool results by rule. Keep thinking blocks when cache hits matter.

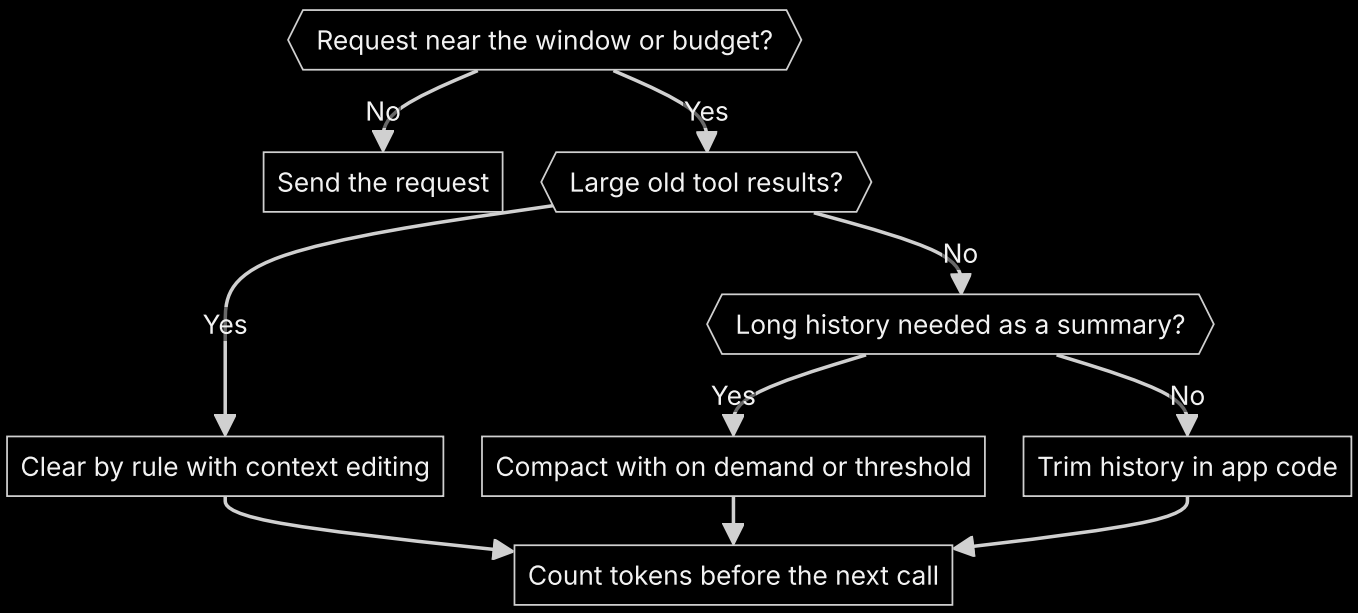
9. Trap

A second summary call on every turn looks thorough, but a rule-based clear needs no extra call.

10. Not in source

- Platform support for context editing beyond the Claude API: Not in source.
- Console steps for context editing: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

D2.5 Reuse and prompt caching

Domain D2, Claude models, prompting, and context engineering. Task statement: Apply prompt reuse and caching to cut cost and latency. (Inference: wording from secondary sources)

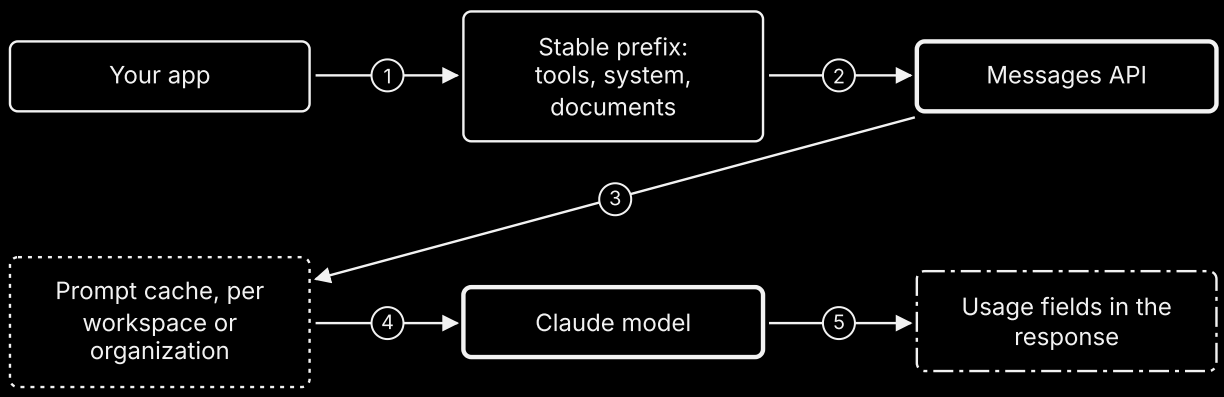
Prompt caching

1. Introduction and first principle

Pay full price for a stable prefix once, then read it at a fraction of the price while it stays identical.

- The default cache lifetime is 5 minutes.[1][2]
- A 5-minute cache write costs 1.25 times the base input price.[1][2]
- A 1-hour cache write costs 2 times the base input price.[1][2]
- A cache read costs 0.1 times the base input price on Claude Haiku 5.5. It costs 0.05 times on Claude Opus 5.5 and Claude Sonnet 5.5, and 0.025 times on Claude Fable 5.1.[1][2]
- A request can define up to 4 cache breakpoints.[1][2]
- The lookback window checks at most 20 block positions per breakpoint.[1][2]
- The minimum cacheable prompt length is 512 tokens for Claude Fable 5.1, Claude Opus 5.5, Claude Sonnet 5.5, and Claude Haiku 5.5.[1][2]
- At 0.1 times the read price, the cache pays off after one read for a 5-minute write and after two reads for a 1-hour write.[1]

2. Architecture



Schematic drawn from docs. Arrows: 1. build request. 2. cache_control on last stable block. 3. look up prefix hash. 4. hit: read, miss: write. 5. cache read and write token counts. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Put stable content first: tools, then system, then messages.
2. Add cache_control to the last block that stays the same across requests.
3. The API computes a hash of the prefix up to that breakpoint.
4. On a hit, the API reads the cached prefix and processes only the new tokens.
5. On a miss, the API writes the prefix to the cache for later requests.

6. Each cache read refreshes the lifetime at no extra cost.
7. Check cache_read_input_tokens and cache_creation_input_tokens to confirm hits.

4. Pros and cons

Pros

- Cuts input cost for repeated prefixes.
- Cuts processing time for long repeated prefixes.
- Needs no extra infrastructure. The API manages the cache.

Cons

- A cache write costs more than a normal input token.
- Any change at or before the breakpoint causes a miss.
- Prompts below the minimum length are not cached, and the API returns no error.

5. Use when, do not use when

Use when

- Many requests share a long system prompt, tool set, or document.
- A multi-turn conversation grows and reuses its history.
- Requests arrive often enough to hit the cache inside the TTL.

Do not use when

- The prefix changes on every request, for example a timestamp at the top.
- The prompt is shorter than the minimum cacheable length for the model.
- Requests arrive less often than the TTL and the 1-hour write price exceeds the saving.

6. Constraint families

Latency

Cache long documents that many requests read. Combination when one choice is not enough. Pre-warm the cache with max_tokens 0 to remove the cache-miss latency on the first user request.

Cost

Cache the stable prefix with the default 5-minute TTL. Combination when one choice is not enough. Add a 1-hour TTL when calls arrive less often than every 5 minutes but more often than every hour.

Quota

Use cache hits to raise rate limit headroom, because cache hits are not deducted against your rate limit. Combination when one choice is not enough. For parallel requests, wait for the first response to start before you send the rest.

Isolation

Use separate workspaces for tenants that must not share a cache. Combination when one choice is not enough. On Amazon Bedrock and Claude Models on Google Cloud (Google managed), isolation is at organization level only.

Scale

Use up to 4 breakpoints to cache sections that change at different rates. Combination when one choice is not enough. Add a second breakpoint when a conversation grows 20 or more blocks past the last write.

Operations

Track cache read and write token counts per request. Combination when one choice is not enough. Alert when the read count drops to zero after a prompt change.

7. Native cloud services

Amazon Bedrock

Bedrock keeps organization-level cache isolation. The AWS Bedrock documentation sets the per-model minimums and usage-field names.[1][2]

Claude Models on Google Cloud (Google managed)

Cache isolation is at organization level, not workspace level. The 512-token minimum for current models applies on this platform.[1][2]

Claude Platform on AWS

Cache isolation is at workspace level, the same as the Claude API. The 1-hour TTL is available.[1][2]

Microsoft Foundry

Cache isolation is at workspace level, the same as the Claude API. The 1-hour TTL is available.[1][2]

8. Exam domain and task statement

D2.5. Expect a scenario with a large repeated prefix and a cost or latency limit. The credited answer moves the stable content first and caches it. (Inference)

If both cache token counts are zero, the prompt was below the minimum or the prefix changed.

Put the breakpoint on the last block whose prefix is identical across the requests that must share the cache.

9. Trap

A custom response cache in front of the API looks cheaper. But it adds a component, and it cannot reuse a prefix for a new question (Inference).

10. Not in source

- Bedrock per-model minimum cacheable length. The Claude docs point to the AWS docs. Not in source.

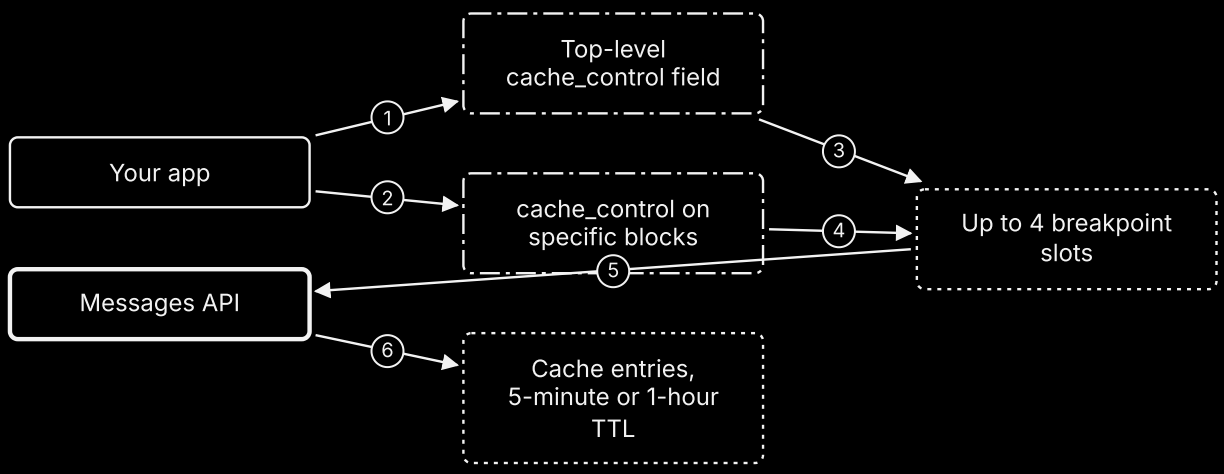
Automatic caching versus explicit breakpoints

1. Introduction and first principle

Automatic caching places one breakpoint for you, and explicit breakpoints fix each cache point by hand.

- Automatic caching applies the cache breakpoint to the last cacheable block.[1][2]
- Automatic caching uses a 5-minute TTL by default.[1][2]
- A ttl value of 1h sets a 1-hour TTL at 2 times the base input price.[1][2]
- Automatic caching uses one of the 4 available breakpoint slots.[1][2]
- In a request with mixed TTLs, 1-hour cache entries must appear before 5-minute cache entries.[1][2]
- Automatic caching is available on every platform except the legacy Amazon Bedrock integration for Claude Opus 4.6 and earlier.[1][2]
- An explicit cache_control on the last block with a different TTL makes automatic caching return a 400 error.[1][2]

2. Architecture



Schematic drawn from docs. Arrows: 1. automatic: one field. 2. explicit: one field per block. 3. uses one slot. 4. uses one slot each. 5. breakpoints in request. 6. read on match, write on miss. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Add one top-level cache_control field to start with automatic caching.

2. The system places one breakpoint on the last cacheable block.
3. The automatic breakpoint moves forward as the conversation grows.
4. Place cache_control on a content block to set an explicit breakpoint.
5. Each explicit breakpoint uses one of the 4 available slots.
6. Automatic caching uses one slot when you combine it with explicit breakpoints.
7. The lookback checks at most 20 positions for each breakpoint.

4. Pros and cons

Pros

- Automatic caching needs one field and no manual breakpoint management.
- Explicit breakpoints cache sections that change at different rates.
- Explicit breakpoints can cache a system prompt while automatic caching handles the conversation.

Cons

- Automatic caching sets the breakpoint on the last cacheable block only.
- Explicit breakpoints need edits when the position of stable content moves.
- Four explicit breakpoints leave no slot, and automatic caching then returns a 400 error.
- If the last block has an explicit cache_control with a different TTL, automatic caching returns a 400 error.

5. Use when, do not use when

Use when

- A multi-turn conversation grows and each turn adds fewer than 20 blocks.
- Sections such as tools, a system prompt, and a document change at different rates.
- The team wants one setting that works without breakpoint management.

Do not use when

- The request already defines 4 explicit block breakpoints.
- The target is the legacy Amazon Bedrock integration for Claude Opus 4.6 and earlier.
- Each turn adds 20 or more blocks past the last write and no second breakpoint exists.

6. Constraint families

Cost

Set the cache TTL to 1 hour with the ttl field in cache_control. Combination when one choice is not enough. When you mix TTLs, put 1-hour entries before any 5-minute entries.

Governance

Document which breakpoint mode each service uses.

Scale

Start with automatic caching for multi-turn conversations. Combination when one choice is not enough. Add an explicit breakpoint near the write point when a turn adds 20 or more blocks.

Operations

Check the cache read and write token counts after each change to the breakpoint setup. Combination when one choice is not enough. Use cache diagnostics to find where the prompt prefix diverged.

7. Native cloud services

Amazon Bedrock

Automatic caching works on Amazon Bedrock. The legacy Bedrock integration for Claude Opus 4.6 and earlier returns a 400 error for a top-level cache_control field.[12]

Claude Platform on AWS

Automatic caching is available on this platform.[12]

Claude Models on Google Cloud (Google managed)

Automatic caching is available on this platform.[12]

Microsoft Foundry

Automatic caching is available on this platform.[12]

8. Exam domain and task statement

D2.5. Expect the scenario to ask which caching mode fits a conversation that gets longer or a fixed set of sections. (Inference)

If a request returns a 400 error after a cache change, count the explicit breakpoints first.

Use automatic caching for conversations that get longer and explicit breakpoints for sections that change at different rates.

9. Trap

A top-level field on a request that already has 4 explicit breakpoints looks like extra protection, but the request fails with a 400 error.

10. Not in source

- Console steps for automatic caching: Not in source.
- Per-platform TTL prices beyond the pricing table: Not in source.

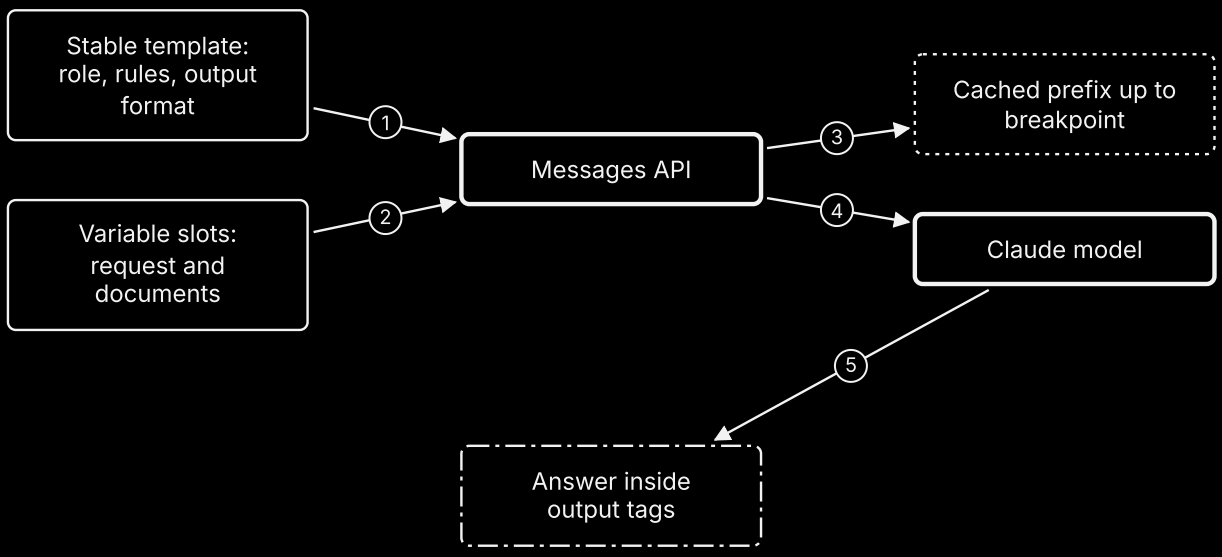
Prompt reuse with templates

1. Introduction and first principle

A template fixes the stable instructions once and fills only the variable slots for each request.

- A cache hit requires 100 percent identical prompt segments up to the cache_control block.[12]
- Include 3 to 5 examples in examples tags for best results.[10]
- Queries placed at the end of a long prompt can improve response quality by up to 30 percent in tests.[10]
- Put long-form data near the top of the prompt for inputs of 20k or more tokens.[10]
- Custom templates are one documented way to define an exact output format.[22]
- The ticket routing guide inserts a variable into request tags in its prompt template.[23]

2. Architecture



Schematic drawn from docs. Arrows: 1. stable text first. 2. variable text last. 3. breakpoint on last stable block. 4. full request. 5. tagged answer. Box line style: heavy line is a Claude service, thin line is your system, long dash is a cloud provider, dots are a data store, dash-dot is a control.

3. How it works

1. Write the stable instructions once as a template with fixed text.
2. Wrap each input type in its own XML tag, such as request or document.
3. Insert the variable content into its slot after the stable text.
4. Place the cache breakpoint on the last stable block of the template.
5. Ask for the answer inside output tags so your code can extract it.
6. Change the template text only with a new version and a new evaluation run.
7. A changed prefix misses the cache, so check the hit rate after each version.

4. Pros and cons

Pros

- Templates separate instructions from data, as the best practices page recommends.
- A fixed prefix supports cache hits across many requests.
- Tagged slots make the output easy to parse.

Cons

- Variable text after the breakpoint is billed at the base input price.
- A template that changes often keeps the cache cold.
- A slot can change the output format, so each template needs its own evaluation.

5. Use when, do not use when

Use when

- Many requests share the same instructions and output format.
- Documents or requests vary, but the rules stay the same.
- Several teams reuse one approved instruction block.

Do not use when

- Each request needs a different structure and a different set of rules.
- The first block changes on every request, for example a timestamp.
- The template text changes for every user, so no prefix repeats.

6. Constraint families

Cost

Keep the template prefix stable and the variable slot last. Combination when one choice is not enough. Add a 1-hour TTL when batches arrive with gaps longer than 5 minutes.

Isolation

Use separate workspaces for tenants that must not share a cache. Combination when one choice is not enough. On Amazon Bedrock and Claude Models on Google Cloud (Google managed), the cache is shared across the organization, so separate tenants another way (Inference).

Governance

Store each template with a named version and an owner. Combination when one choice is not enough. Review each template change against the evaluation set before release (Inference).

Operations

Keep one live template version for each use case. Combination when one choice is not enough. Use cache diagnostics when the hit rate falls after a template change.

7. Native cloud services

Amazon Bedrock

Bedrock uses organization-level cache isolation, so templates share caches inside one organization.[1][2]

Microsoft Foundry

Caches are isolated by workspace, the same as the Claude API.[1][2]

Claude Models on Google Cloud (Google managed)

This platform uses organization-level cache isolation, so templates share caches inside one organization.[1][2]

Claude Platform on AWS

Caches are isolated by workspace, the same as the Claude API.[1][2]

8. Exam domain and task statement

D2.5. Expect the scenario to ask how to reuse one instruction block across many calls without a custom cache. (Inference)

When a template change drops the hit rate to zero, check the block order before you change the TTL.

Put stable template text first, variable slots last, and version every change to the template.

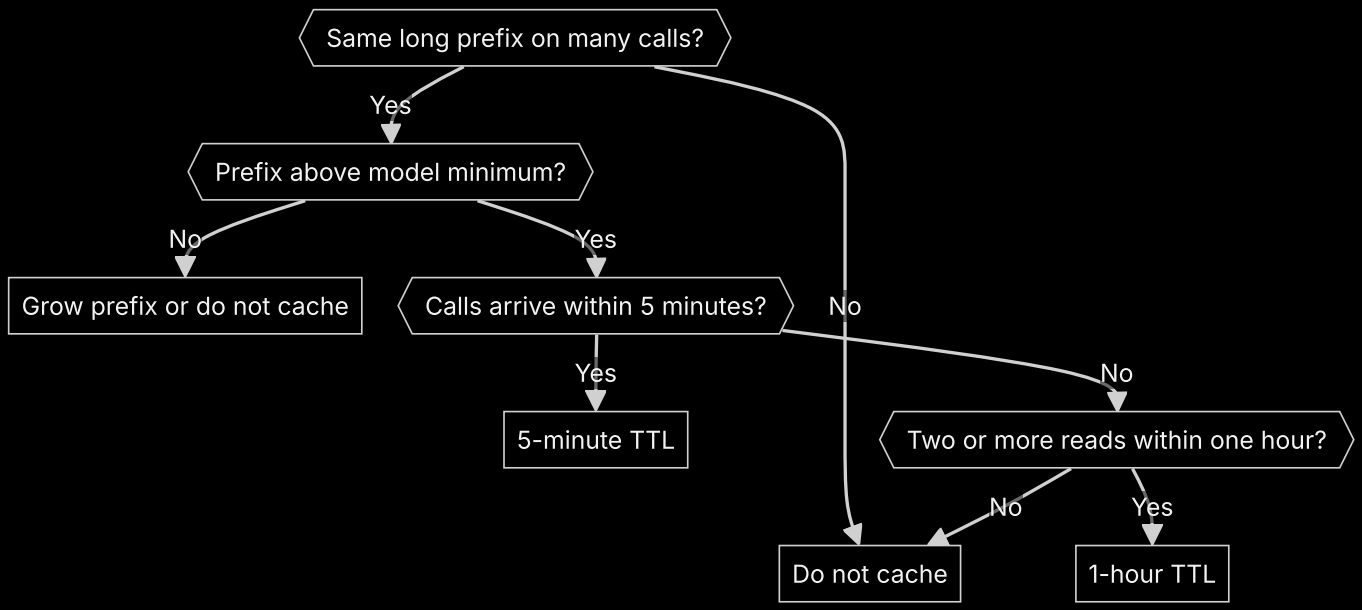
9. Trap

A rebuilt full prompt with a timestamp in each request looks harmless, but it breaks the cached prefix on every call.

10. Not in source

- A built-in prompt template feature in the Claude Console: Not in source.
- Limits on the number of stored template versions: Not in source.
- Console steps for prompt templates: Not in source.

Decision flow



Unit items not in source

- Console steps: Not in source.

Sources

1. Pricing. https://platform.claude.com/docs/en/about-claude/pricing
2. Models overview. https://platform.claude.com/docs/en/models/overview
3. Choosing the right model. https://platform.claude.com/docs/en/about-claude/models/choosing-a-model
4. Claude Models on Google Cloud (Google managed). https://platform.claude.com/docs/en/build-with-claude/claude-models-on-google-cloud/google-managed
5. Claude in Microsoft Foundry. https://platform.claude.com/docs/en/build-with-claude/claude-in-microsoft-foundry
6. Claude Platform on AWS. https://platform.claude.com/docs/en/build-with-claude/claude-platform-on-aws
7. Effort. https://platform.claude.com/docs/en/build-with-claude/effort
8. Thinking. https://platform.claude.com/docs/en/build-with-claude/thinking
9. Claude Haiku 5.5. https://platform.claude.com/docs/en/models/haiku-5-5/overview
10. Prompting best practices. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
11. Mitigate jailbreaks and prompt injections. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks
12. Prompt caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching
13. Structured outputs. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
14. Prompt engineering overview. https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
15. Context windows. https://platform.claude.com/docs/en/build-with-claude/context-windows
16. Token counting. https://platform.claude.com/docs/en/build-with-claude/token-counting
17. Compaction on demand. https://platform.claude.com/docs/en/build-with-claude/compaction-on-demand
18. Compaction at a token threshold. https://platform.claude.com/docs/en/build-with-claude/compaction-threshold
19. Compaction overview. https://platform.claude.com/docs/en/build-with-claude/compaction
20. Compaction that keeps recent turns. https://platform.claude.com/docs/en/build-with-claude/compaction-keep-recent-turns
21. Context editing. https://platform.claude.com/docs/en/build-with-claude/context-editing
22. Increase output consistency. https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/increase-consistency
23. Ticket routing. https://platform.claude.com/docs/en/about-claude/use-case-guides/ticket-routing
24. Cache diagnostics. https://platform.claude.com/docs/en/build-with-claude/cache-diagnostics